

XOR Subsequence Intuition

The key observation is that **we only care whether the XOR is zero or non-zero**, not its actual value.

Step 1: What happens if total XOR is non-zero?

Suppose:

```
nums = [2, 3, 4]
```

Total XOR:

```
2 ^ 3 ^ 4 = 5
```

Since it's non-zero, we can simply take **all n elements**.

So answer = n.

Step 2: What if total XOR is zero?

Suppose:

```
nums = [1, 2, 3]
```

Total XOR:

```
1 ^ 2 ^ 3 = 0
```

So taking all 3 elements doesn't work.

Now we ask:

Can we remove just **one element** and make the XOR non-zero?

If we remove 1:

```
2 ^ 3 = 1
```

Non-zero 

Therefore answer = n - 1.

But there is one important case.

Step 3: What if every element is zero?

```
nums = [0, 0, 0]
```

Any subsequence will have:

```
0 ^ 0 ^ 0 = 0
```

Even if we take only one:

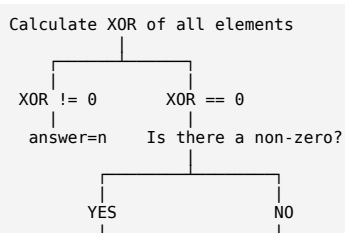
```
0
```

Still zero.

So answer = 0.

The complete intuition

There are only **3 cases**:



answer=n-1 answer=0

Why does removing one non-zero element work?

If:

total XOR = 0

and we remove x , then the remaining XOR is:

$0 \oplus x = x$

because:

$a \oplus a = 0$
 $0 \oplus a = a$

So if **any** $x \neq 0$ **exists**, removing that element gives us a non-zero XOR.

Therefore:

```
C++
if (totalXor != 0)
    return n;

if (there is at least one non-zero element)
    return n - 1;

return 0;
```

Complexity

We only scan the array once:

- **Time:** $O(n)$
- **Space:** $O(1)$

The beautiful part of this problem is that you **don't need DP, subsequence generation, or XOR subsets**. The XOR property reduces the whole problem to checking the total XOR and whether a non-zero element exists.

```
C++
class Solution {
public:
    int longestSubsequence(vector<int>& nums) {
        int n = nums.size();
        int totalXor = 0;
        bool hasNonZero = false;

        for (int x : nums) {
            totalXor ^= x;

            if (x != 0) {
                hasNonZero = true;
            }
        }

        // If XOR of all elements is non-zero,
        // we can take the entire array.
        if (totalXor != 0) {
            return n;
        }

        // Total XOR is zero, but removing one non-zero
        // element makes the remaining XOR non-zero.
        if (hasNonZero) {
            return n - 1;
        }

        // All elements are zero.
        return 0;
    }
};
```

In short

For:

[1, 2, 3]

$1 \oplus 2 \oplus 3 = 0$

There is a non-zero element $\rightarrow$ remove one $\rightarrow$ answer $3 - 1 = 2$.

For:

```
[2, 3, 4]
```

```
2 ^ 3 ^ 4 = 5
```

Non-zero → take everything → answer 3.

For:

```
[0, 0, 0]
```

No non-zero element exists → answer 0.

Time: $O(n)$

Space: $O(1)$

ChatGPT can make mistakes. Check important info.